PARALLEL COMPUTING

Performance, Speedup, Scalability and Efficiency

Topics covered in this guide:

Serial vs. parallel execution, speedup, and efficiency

Why perfect speedup is rarely achieved: load imbalance, communication, serialization, overhead

Amdahl's Law and the limits of parallel scaling

Strong scaling vs. weak scaling

Performance modeling: computation, memory, and communication time

Latency, bandwidth, and overlapping operations

CPU-bound, memory-bound, and communication-bound classification

Arithmetic intensity, cache blocking, and BLAS/LAPACK/ScaLAPACK

A. Basic Performance Concepts

Serial and Parallel Execution

A program can be executed using one processor/core (**serial execution**) or multiple processors/cores (**parallel execution**).

Notation:

- T_1 = execution time using 1 core
- T_p = execution time using p cores/processors
- p = number of cores/processors

The goal of parallel computing is generally to reduce T_p by dividing the work among multiple processors.

Ideal Case — Perfect / Linear Speedup

$$T_p = T_1 / p$$

Ideal parallel execution time

EXAMPLE

If a program takes 100 seconds on 1 core: T_1 = 100 seconds.

Using 4 cores: T_p = 100 / 4 = **25 seconds**.

This is called **perfect** or **linear speedup**.

Speedup

Speedup tells us how much faster the parallel version is compared with the serial version.

$$S_p = T_1 / T_p$$

S_p = speedup | T_1 = serial time | T_p = parallel time with p processors

EXAMPLE

T_1 = 100 seconds, T_p = 25 seconds

S_p = 100 / 25 = **4**

The program achieved a speedup of 4 — it is 4 times faster than the serial version.

Perfect or Linear Speedup

$$S_{ideal} = p$$

Cores (p)	Ideal Speedup

2	2
4	4
8	8
16	16

However, perfect speedup is rarely achieved in real programs.

B. Why Perfect Speedup Is Rare

There are several reasons why adding more processors does not always give proportional performance improvement.

1. Load Imbalance

Load imbalance occurs when different processors receive different amounts of work.

> **EXAMPLE**
>
> Processor 1 → 10 seconds of work
>
> Processor 2 → 10 seconds of work
>
> Processor 3 → 20 seconds of work
>
> Processors 1 and 2 finish early and sit idle while Processor 3 keeps working.

The total execution time is determined by the processor that finishes **last**. Load imbalance is a key reason processors can remain idle while waiting for others.

2. Communication Overhead

Processors often need to communicate or exchange data — for example, Processor 1 calculates part A, Processor 2 calculates part B, and Processor 1 must send data to Processor 2 before it can continue.

This communication takes time, so adding processors does not automatically make the program faster. Communication can limit scalability when it cannot be overlapped with computation.

3. Non-Parallelizable Sections

Some operations cannot be executed simultaneously — e.g., if A must be calculated before B, and B before C, then A → B → C must execute sequentially.

Mutual dependencies and shared resources are common reasons why some operations must be serialized.

4. Parallelization Overhead

Creating and managing threads/processes also requires time — creating threads, assigning work, synchronizing, communicating, and destroying/joining threads.

This extra work is called **parallelization overhead**. As a result:

$$\textbf{Parallel execution time} \neq \textbf{Serial time / Number of processors}$$

Creating threads or processes is not free — it has startup overhead.

C. Efficiency

Speedup tells us how much faster the program became. **Efficiency** tells us how effectively the processors are being used.

$$E_p = S_p / p \quad \text{or} \quad E_p = T_1 / (p \times T_p)$$

Where E_p = efficiency, S_p = speedup, p = number of processors, T_p = parallel execution time.

$$E_{ideal} = 1 = 100\%$$

Ideal efficiency

EXAMPLE

p = 4, T_1 = 100 s, T_p = 30 s

S_p = 100 / 30 = 3.33

E_p = 3.33 / 4 ≈ **0.833 (83.3%)**

The system achieved about 83.3% of ideal processor utilization.

D. Speedup vs. Efficiency

	Speedup (S■)	Efficiency (E■)
Formula	S■ = T■ / T■	E■ = S■ / p
Question answered	How many times faster is the parallel program?	How close are we to the ideal speedup?

EXAMPLE

8 processors, Speedup = 6

Efficiency: E = 6 / 8 = **0.75 (75%)**

E. Amdahl's Law

Amdahl's Law explains an important limitation of parallel computing: a program may contain both **serial work** and **parallelizable work**.

$$T_1 = W_{ser} + W_{par}$$

Where W_{ser} = non-parallelizable serial work, W_{par} = parallelizable work.

With p processors:

$$T_p \geq W_{ser} + W_{par} / p$$

Serial Fraction

Let f = fraction of the total work that is serial. The maximum possible speedup is limited by:

$$S_{max} = 1 / f$$

Even with an unlimited number of processors, the serial portion cannot be parallelized — it becomes the limiting factor. This is one of the most important ideas in the lecture.

EXAMPLE

Suppose 10% of a program is serial: f = 0.10

Maximum speedup: S_{max} = 1 / 0.10 = **10**

Even with infinitely many processors, the maximum speedup is 10.

F–G. Strong Scaling and Weak Scaling

F. Strong Scaling

"How does execution time change when we increase the number of processors while keeping the total problem size fixed?"

EXAMPLE

Problem size = 1,000,000 elements (fixed)

Run with 1, 2, 4, 8 processors — only the processor count changes.

G. Weak Scaling

"How does execution time change when both the number of processors and the problem size increase at the same rate?"

EXAMPLE

1 processor → 1 million elements

2 processors → 2 million elements

4 processors → 4 million elements

8 processors → 8 million elements

The amount of work **per processor** stays approximately constant.

This concept is associated with **Gustafson and Barsis**.

H. Strong vs. Weak Scaling — Comparison

	Strong Scaling	Weak Scaling
Problem size	Fixed	Increases
Processors	Increase	Increase
Main question	Can we solve the same problem faster?	Can we handle a larger problem while keeping runtime ~constant?

This distinction is very important for parallel-computing exams.

I. Performance Model

The lecture divides total execution time into several components:

$$T_{total} = T_F + T_M + T_C$$

- T_F = computation/flop time
- T_M = memory time
- T_C = communication time

Processor activity is generally described in terms of computation, data access, communication, and — in some environments — disk I/O.

Computation Time

Let f = total number of floating-point operations, t_f = time per floating-point operation:

$$T_F = f \times t_f$$

Memory Time

Let m = number of memory elements moved, t_m = time per slow-memory access:

$$T_M \approx m \times t_m$$

Communication Time

Let c = total number of bytes communicated, t_c = average time to communicate one byte:

$$T_C = c \times t_c$$

Putting it together:

$$T_{total} = f \times t_f + m \times t_m + c \times t_c$$

J. Latency and Bandwidth

Two important concepts in communication performance:

Latency (α)

Latency is the time before transmission of the first byte begins to arrive at the destination. Unit: seconds. Low latency means the initial delay is small.

Bandwidth (β)

Bandwidth describes how much data can be transferred per unit of time. Unit: bytes/second. High bandwidth means data can be transferred at a high rate.

Communication Time Formula

For a message of size D:

$$\textbf{Transmission time} = \alpha + D / \beta$$

α = *latency* | *D = amount of data* | β = *bandwidth*

Communication performance depends on both the initial delay and the data-transfer rate.

K. Memory Access vs. Network Communication

The same basic latency/bandwidth idea applies to memory access:

Network	$T = \alpha + D / \beta$
Memory	Similar model, but with different (smaller) values

Memory latency is typically much lower because memory is physically closer and uses faster links. **Prefetching** is discussed as a way of hiding memory latency.

L. Overlapping Operations

Different activities do not always have to happen completely one after another — they can sometimes overlap.

> **EXAMPLE**
>
> **Without overlap:** Computation → Memory Access → Computation → Memory Access
>
> **With overlap:** Computation + Memory Access (simultaneously)

This can reduce total execution time. Approaches include:

- **Prefetching** — overlap computation and memory access
- **Non-blocking communication** — overlap computation and communication
- **Separate computation/communication threads** — use different threads for different activities

M. CPU-Bound, Memory-Bound, and Communication-Bound

This is a very important classification of program behavior.

Classification	Dominant Term	Example
CPU-bound	T■■■■■ ≈ T■	Dense matrix computation
Memory-bound	T■■■■■ ≈ T■	Bioinformatics applications
Communication-bound	T■■■■■ ≈ T■	Graph traversal, sparse matrix computation

CPU-bound: computation time dominates; the CPU performs close to its peak.

Memory-bound: memory-access time dominates; improving data locality and access patterns can help.

Communication-bound: communication time dominates; these applications spend significant time exchanging data.

N. Arithmetic Intensity

Arithmetic intensity measures how much computation is performed relative to memory movement — a FLOP-to-byte ratio.

$$q = f / m$$

q = arithmetic intensity | f = floating-point operations | m = data moved from slow memory

A higher q generally means more computation is performed for each unit of data moved. Arithmetic intensity is an important factor in algorithm efficiency.

O. Machine Balance

The performance model can be written as:

$$T_{total} = f \times t_f \times (1 + (t_m/t_f) \times (1/q))$$

Machine balance relates the cost of memory access to the cost of computation. The lecture states that:

$$q \geq t_m / t_f$$

...is needed to obtain at least half of peak speed

Low q	Few computations per memory access → memory has a large effect → performance tends toward mem
High q	Many computations per memory access → memory cost becomes relatively less important → performa

P. Case Study: Matrix-Vector Multiplication

Consider: y = y + A × x

EXAMPLE

```
for i = 1:n
  for j = 1:n
    y(i) = y(i) + A(i,j) × x(j)
```

Number of floating-point operations:

$$f = 2n^2$$

The derived arithmetic intensity, for double precision (8 bytes per double), is approximately:

$$q \approx 2 / 8 = 0.25$$

Conclusion: matrix-vector multiplication is generally memory-bound. The amount of computation per amount of data moved is relatively small, so improving memory access can be more useful than simply increasing CPU computation capability.

Q–R. Cache, Memory Locality, and Cache Blocking

Q. Cache and Memory Locality

The basic memory hierarchy:

EXAMPLE

CPU

↓

Fast memory / Cache

↓

Slow memory

If data is already in cache, accessing it is much faster than retrieving it from slower memory — which is why algorithms should try to reuse data while it is in fast memory.

R. Cache Blocking (Tiling)

Cache blocking, also called tiling, reorganizes computation so that data can be reused while it is in cache. Instead of processing a large data structure in a way that repeatedly accesses slow memory, it is divided into smaller blocks. This reduces cache misses and improves data reuse.

Main idea:

EXAMPLE

Large problem → divide into smaller blocks → place useful blocks in cache →

perform more computation using cached data → reduce slow-memory accesses

S–T. Matrix Multiplication and Blocking

S. Matrix Multiplication

Consider: $C = C + A \times B$

EXAMPLE

```
for i = 1 to n
  for j = 1 to n
    for k = 1 to n
      C(i,j) = C(i,j) + A(i,k) × B(k,j)
```

$$f = 2n^3$$

Matrix multiplication can have much higher arithmetic intensity than matrix-vector multiplication when properly organized.

T. Blocked / Tiled Matrix Multiplication

A, B, and C are divided into smaller submatrices called blocks, and the algorithm works on these blocks while keeping them in fast memory. For large n:

$$q \approx b$$

b = block size

Increasing the block size can increase arithmetic intensity — as long as the required blocks fit in cache.

U. Why Block Size Matters

The three blocks A(i,k), B(k,j), and C(i,j) must fit in fast memory. If M_{fast} = size of fast memory:

$$3b^2 \leq M_{fast}$$

Therefore:

$$b \leq \sqrt{(M_{fast} / 3)}$$

This means the block cannot simply be made arbitrarily large — it must be small enough to fit in cache.

V. BLAS — Basic Linear Algebra Subroutines

BLAS is an industry-standard interface for linear algebra operations, divided into three major levels:

Level	Operation Type	Example	Computational Intensity
BLAS1	Vector operations	Dot product, SAXPY (y = αx + y)	Low
BLAS2	Matrix-vector operations	Matrix-vector multiplication	Higher than BLAS1
BLAS3	Matrix-matrix operations	Matrix multiplication	Much higher — can be far faster than BLAS2

W. LAPACK and ScaLAPACK

LAPACK (Linear Algebra PACKage) is built on top of BLAS and provides common linear algebra operations such as:

- Solving systems of linear equations
- Least-squares problems
- Eigenvalue problems
- SVD problems

ScaLAPACK is a distributed-memory implementation of LAPACK. Both are examples of software built around optimized linear algebra operations.

X. Most Important Concepts to Remember

Concept	Formula	Meaning
Speedup	S_p = T₁ / T_p	How many times faster the parallel program is
Efficiency	E_p = S_p / p	How close the program is to ideal speedup
Perfect Speedup	S_{ideal} = p	Ideal speedup equals the number of processors
Amdahl's Law	S_{max} = 1 / f	Serial work limits maximum speedup (f = serial fraction)
Strong Scaling	—	Fixed problem size, increase processors: how much faster can we solve the same p
Weak Scaling	—	Problem size increases with processors: can we solve a larger problem without muc
Latency (α)	—	Initial communication delay
Bandwidth (β)	—	Rate of data transfer
Communication Time	$T = \alpha + D/\beta$	Time to transmit a message of size D
Arithmetic Intensity	$q = f/m$	Higher q $\rightarrow$ more computation relative to memory movement
CPU-bound	T_{total} ≈ T_F	Computation dominates
Memory-bound	T_{total} ≈ T_M	Memory access dominates
Communication-bound	T_{total} ≈ T_C	Communication dominates
Cache Blocking	—	Divide computation into blocks so data is reused efficiently in fast memory
BLAS Levels	—	BLAS1 = vector, BLAS2 = matrix-vector, BLAS3 = matrix-matrix

Y. Formula Sheet

Quantity	Formula
Serial execution time	**T■ = W■■■ + W■■■**
Parallel execution time	**T■ ≥ W■■■ + W■■■/p**
Speedup	**S■ = T■/T■**
Efficiency	**E■ = S■/p**
Ideal speedup	**S_ideal = p**
Ideal efficiency	**E_ideal = 1 = 100%**
Amdahl maximum speedup	**S_max = 1/f**
Total performance model	**T_total = T_F + T_M + T_C**
Detailed model	**T_total = f×t_f + m×t_m + c×t_c**
Communication time	**T_comm = α + D/β**
Arithmetic intensity	**q = f/m**
Matrix-vector multiplication	**f = 2n²**
Matrix multiplication	**f = 2n³**
Blocked matrix multiplication	**q ≈ b**
Cache requirement (3 blocks)	**3b² ≤ M_fast**

Z. Big Picture — The Connected Story

The lecture can be understood as one connected narrative flowing from motivation to optimization:

1	Parallel program → use multiple processors → try to reduce execution time
2	Measure SPEEDUP, then compare it with the number of processors
3	Calculate EFFICIENCY — why isn't it 100%?
4	Causes: load imbalance · communication · serial sections · parallelization overhead
5	AMDAHL'S LAW → serial work limits achievable speedup
6	Analyze where execution time is spent: computation / memory / communication
7	Classify the program: CPU-bound / memory-bound / communication-bound
8	Use arithmetic intensity and locality to guide optimization
9	Optimize memory access with cache blocking / tiling
10	Result: higher arithmetic intensity → better overall performance

---

End of notes.